playground

February 5, 2026

[88]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import morethemes as mt
mt.set_theme('economist')
import warnings
warnings.filterwarnings('ignore')
import yfinance as yf
import fredapi as fp
```

1 Organization

Data loading, CPI adjustments, and dataset preparation.

[89]:
```
# Read professor-only UC salary CSV files
import os
import glob

# Get the directory of the current notebook
notebook_dir = '/Users/joshkenworthy/Library/CloudStorage/
 ↪OneDrive-UCLAITServices/UC Salary Data'

# Find professor-only CSVs
csv_files = sorted(glob.glob(os.path.join(notebook_dir, 'uc_professors_*.csv')))

# Read each CSV and create dataframes with year-based names
for csv_file in csv_files:
    filename = os.path.basename(csv_file)
    year = filename.split('_')[-1].replace('.csv', '')
    year_short = year[-2:]

    df_name = f'df{year_short}'
    globals()[df_name] = pd.read_csv(csv_file)

    # Compatibility alias for existing analysis logic
```

```
    if 'TotalPay' not in globals()[df_name].columns and 'TotalWages' in␣
↪globals()[df_name].columns:
        globals()[df_name]['TotalPay'] = globals()[df_name]['TotalWages']
```

[90]:
```
# Configure pay columns (nominal and real)
pay_col = 'TotalPay'
real_pay_col = 'RealTotalPay'
salary_col = real_pay_col
salary_label = 'TotalPay (CPI-adjusted, 2024$)'

# Load CPI inflation rates and build a CPI index
cpi_path = os.path.join(notebook_dir, 'cpi.csv')
cpi = pd.read_csv(cpi_path)
cpi['Year'] = pd.to_datetime(cpi['observation_date']).dt.year
cpi = cpi.sort_values('Year').reset_index(drop=True)
cpi['CPI_Index'] = 100.0
for i in range(1, len(cpi)):
    cpi.loc[i, 'CPI_Index'] = cpi.loc[i - 1, 'CPI_Index'] * (1 + cpi.loc[i,␣
↪'CPIAUCSL'] / 100.0)

ref_year = int(cpi['Year'].max())
ref_index = float(cpi.loc[cpi['Year'] == ref_year, 'CPI_Index'].iloc[0])
cpi_index = dict(zip(cpi['Year'], cpi['CPI_Index']))

import re

def normalize_col(col):
    return re.sub(r'[^a-z0-9]+', '', col.lower())

def resolve_totalpay_column(df):
    normalized = {col: normalize_col(col) for col in df.columns}
    for col, norm in normalized.items():
        if norm == 'totalpay':
            return col, 'Exact'
    for col, norm in normalized.items():
        if 'totalpay' in norm and 'benefits' not in norm:
            return col, 'Partial'
    for col, norm in normalized.items():
        if norm == 'totalpaybenefits':
            return col, 'Fallback:TotalPayBenefits'
    return None, 'Not Found'

removed_summary = []
pay_components = ['RegularPay', 'OvertimePay', 'LumpSumPay', 'OtherPay']

for year in range(13, 25):
    df_name = f"df{year:02d}"
```

```
    if df_name in globals():
        df = globals()[df_name]
        year_full = 2000 + year
        if pay_col in df.columns:
            resolved_pay_col, resolution = pay_col, 'Exact'
        else:
            resolved_pay_col, resolution = resolve_totalpay_column(df)
            if resolved_pay_col is None and all(col in df.columns for col in␣
↪pay_components):
                df[pay_col] = df[pay_components].fillna(0).sum(axis=1)
                resolved_pay_col, resolution = pay_col, 'Computed from␣
↪components'
        if resolved_pay_col and year_full in cpi_index:
            df = df[df[resolved_pay_col] > 0].copy()
            deflator = ref_index / cpi_index[year_full]
            df[real_pay_col] = df[resolved_pay_col] * deflator
            globals()[df_name] = df
            removed_summary.append({
                'Year': str(year_full),
                'Pay Col': resolved_pay_col,
                'Resolution': resolution
            })
        else:
            removed_summary.append({
                'Year': str(year_full),
                'Pay Col': resolved_pay_col if resolved_pay_col else 'Not␣
↪Found',
                'Resolution': resolution
            })
```

[91]:
```
# Build professor-only subsets (2013-2024) with required columns only
# Uses the same professor definition already implemented in this notebook.
professor_pattern = r'Prof[\s\-]'
required_cols = ['Year', 'EmployerName', 'Position', 'RegularPay', 'TotalWages']

for year in range(2013, 2025):
    df_name = f"df{str(year)[-2:]}"
    if df_name in globals():
        df_year = globals()[df_name]
        prof_subset = df_year[df_year['Position'].str.
↪contains(professor_pattern, case=False, na=False, regex=True)].copy()
        prof_subset['Year'] = year
        uc_df_name = f"uc_professors_{year}"
        globals()[uc_df_name] = prof_subset[required_cols].copy()
```

[92]:
```
# Validate professor subsets before exporting
required_cols = ['Year', 'EmployerName', 'Position', 'RegularPay', 'TotalWages']
professor_pattern = r'Prof[\s\-]'

validation_rows = []
all_ok = True

for year in range(2013, 2025):
    df_name = f"uc_professors_{year}"
    if df_name in globals():
        df = globals()[df_name]
        cols_ok = list(df.columns) == required_cols
        pattern_ok = df['Position'].str.contains(professor_pattern, case=False,␣
 ↪na=False, regex=True).all()
        year_ok = (df['Year'] == year).all()
        row_count = len(df)
        validation_rows.append({
            'Year': year,
            'Rows': row_count,
            'Cols OK': cols_ok,
            'Prof Pattern OK': pattern_ok,
            'Year OK': year_ok
        })
        all_ok = all_ok and cols_ok and pattern_ok and year_ok
    else:
        validation_rows.append({
            'Year': year,
            'Rows': None,
            'Cols OK': False,
            'Prof Pattern OK': False,
            'Year OK': False
        })
        all_ok = False

validation_df = pd.DataFrame(validation_rows)
```

[93]:
```
# Export professor subsets to CSV (run after validation shows All datasets␣
 ↪valid: True)
export_dir = notebook_dir
export_summary = []

for year in range(2013, 2025):
    df_name = f"uc_professors_{year}"
    if df_name in globals():
        df = globals()[df_name]
        output_path = os.path.join(export_dir, f"uc_professors_{year}.csv")
        df.to_csv(output_path, index=False)
```

```
        export_summary.append({
            'Year': year,
            'Path': output_path,
            'Rows': len(df)
        })

export_df = pd.DataFrame(export_summary)
```

2 Analysis: University Breakdown

Descriptive statistics comparing the UC system campuses.

[ ]:
```
# Prepare 2024 data for university analysis
prof_2024 = df24[df24['Position'].str.contains(r'Prof[\s\-]', case=False,␣
 ↪na=False, regex=True)].copy()
prof_2024['Rank'] = prof_2024['Position'].apply(categorize_rank)
prof_2024['TotalPay'] = prof_2024.get('TotalPay', prof_2024.get('TotalWages',␣
 ↪0))

# 1. Total Professor Count by University
univ_counts = prof_2024.groupby('EmployerName').size().
 ↪sort_values(ascending=False).reset_index(name='TotalProfessors')

# 2. Rank Distribution by University (both counts and percentages)
rank_by_univ = pd.crosstab(prof_2024['EmployerName'], prof_2024['Rank'])
rank_by_univ_pct = pd.crosstab(prof_2024['EmployerName'], prof_2024['Rank'],␣
 ↪normalize='columns') * 100
rank_by_univ_pct = rank_by_univ_pct.round(1)

# 3. Salary Quartiles by University
def get_quartiles(series):
    return pd.Series({
        'Q1 (25%)': series.quantile(0.25),
        'Median (50%)': series.quantile(0.50),
        'Q3 (75%)': series.quantile(0.75),
        'Mean': series.mean(),
        'Std Dev': series.std()
    })

salary_quartiles = prof_2024.groupby('EmployerName')['TotalPay'].
 ↪apply(get_quartiles).round(0)

# 4. Compensation by Rank within each University
salary_by_rank_univ = prof_2024.pivot_table(values='TotalPay',␣
 ↪index='EmployerName', columns='Rank', aggfunc='mean').round(0)
```

```
# 5. Identify notable differences (highest/lowest mean by rank across␣
↪universities)
rank_var = prof_2024.groupby(['Rank', 'EmployerName'])['TotalPay'].mean().
↪unstack()
rank_ranges = pd.DataFrame({
    'Highest': rank_var.max(axis=1),
    'Lowest': rank_var.min(axis=1),
    'Range': rank_var.max(axis=1) - rank_var.min(axis=1),
    'Range %': ((rank_var.max(axis=1) - rank_var.min(axis=1)) / rank_var.
↪min(axis=1) * 100).round(1)
})
```

```
                Metric           Value
0  Total Analysis Date            2024
1   Total Universities              10
2     Total Professors           18898
3        Analysis Type  Nominal Salary

                              EmployerName  TotalProfessors
0  University of California, San Francisco             3948
1      University of California, San Diego             3259
2    University of California, Los Angeles             2929
3          University of California, Davis             2730
4         University of California, Irvine             2274
5       University of California, Berkeley             1690
6      University of California, Riverside              986
7     University of California, Santa Cruz              691
8         University of California, Merced              337
9  University of California, Santa Barbara               54

                                  Rank Distribution (Counts)
Assistant  {'University of California, Berkeley': 405, 'U…
Associate  {'University of California, Berkeley': 391, 'U…
Clinical   {'University of California, Berkeley': 21, 'Un…
Full       {'University of California, Berkeley': 868, 'U…
Research   {'University of California, Berkeley': 5, 'Uni…

EmployerName
University of California, Berkeley       Q1 (25%)        149758.0
                                         Median (50%)    205692.0
                                         Q3 (75%)        289099.0
                                         Mean            224817.0
                                         Std Dev         119649.0
University of California, Davis          Q1 (25%)        158540.0
                                         Median (50%)    222273.0
                                         Q3 (75%)        317820.0
                                         Mean            256476.0
                                         Std Dev         149172.0
University of California, Irvine         Q1 (25%)        151325.0
```

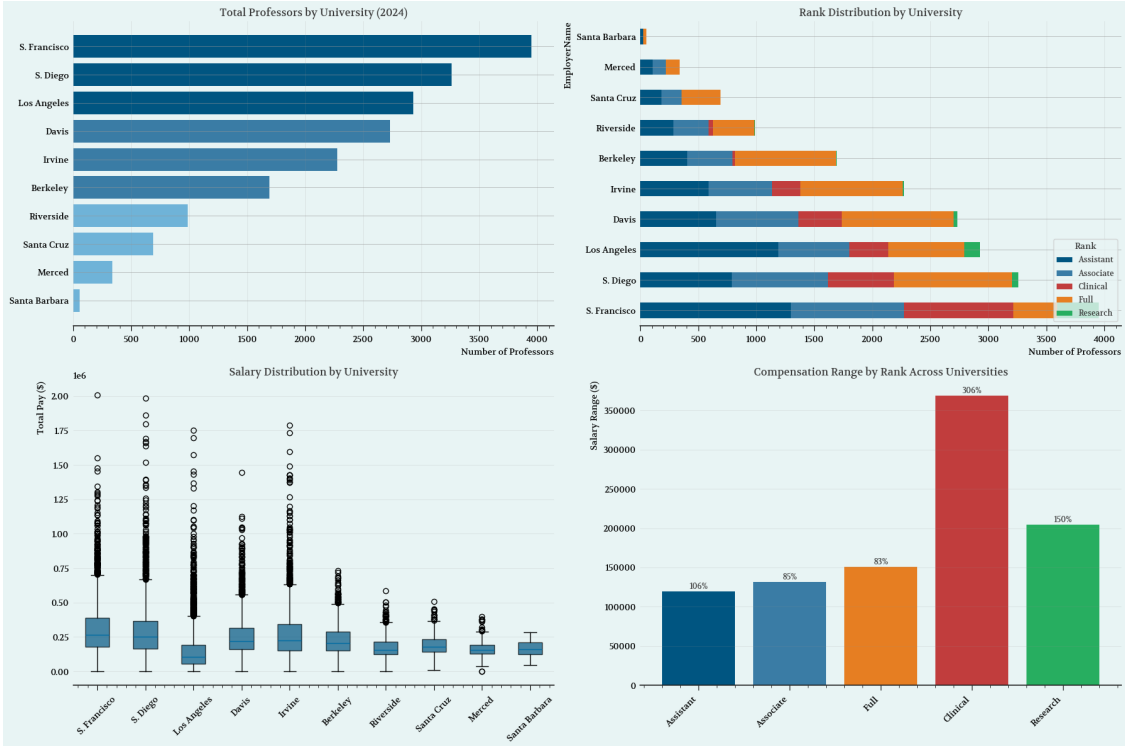
```
                                        Median (50%)    222896.0
                                        Q3 (75%)        344534.0
                                        Mean            277151.0
                                        Std Dev         199982.0
University of California, Los Angeles   Q1 (25%)         54443.0
                                        Median (50%)    106182.0
                                        Q3 (75%)        194300.0
                                        Mean            159219.0
                                        Std Dev         176454.0
University of California, Merced        Q1 (25%)        127967.0
                                        Median (50%)    154633.0
                                        Q3 (75%)        194146.0
                                        Mean            164933.0
                                        Std Dev          58188.0
University of California, Riverside     Q1 (25%)        122450.0
                                        Median (50%)    155700.0
                                        Q3 (75%)        216487.0
                                        Mean            167993.0
                                        Std Dev          88703.0
University of California, San Diego     Q1 (25%)        165111.0
                                        Median (50%)    251350.0
                                        Q3 (75%)        366932.0
                                        Mean            292983.0
                                        Std Dev         199162.0
University of California, San Francisco Q1 (25%)        177325.0
                                        Median (50%)    267290.0
                                        Q3 (75%)        389062.0
                                        Mean            298740.0
                                        Std Dev         193851.0
University of California, Santa Barbara Q1 (25%)        122404.0
                                        Median (50%)    162771.0
                                        Q3 (75%)        211074.0
                                        Mean            164363.0
                                        Std Dev          60688.0
University of California, Santa Cruz    Q1 (25%)        142954.0
                                        Median (50%)    177257.0
                                        Q3 (75%)        232976.0
                                        Mean            190955.0
                                        Std Dev          72485.0
Name: TotalPay, dtype: float64

                                Avg Salary by Rank (Nominal)
Assistant  {'University of California, Berkeley': 138240…
Associate  {'University of California, Berkeley': 186300…
Clinical   {'University of California, Berkeley': 120333…
Full       {'University of California, Berkeley': 285018…
Research   {'University of California, Berkeley': 237652…

                 Highest         Lowest          Range  Range %
```

```
Rank
Assistant  231530.078704  112245.829787  119284.248916    106.3
Associate  285942.556237  154302.192182  131640.364055     85.3
Clinical   489146.290984  120332.761905  368813.529079    306.5
Full       332725.359043  181660.907436  151064.451607     83.2
Research   341173.725490  136650.000000  204523.725490    149.7
```

2.1 Visualization: University Breakdown

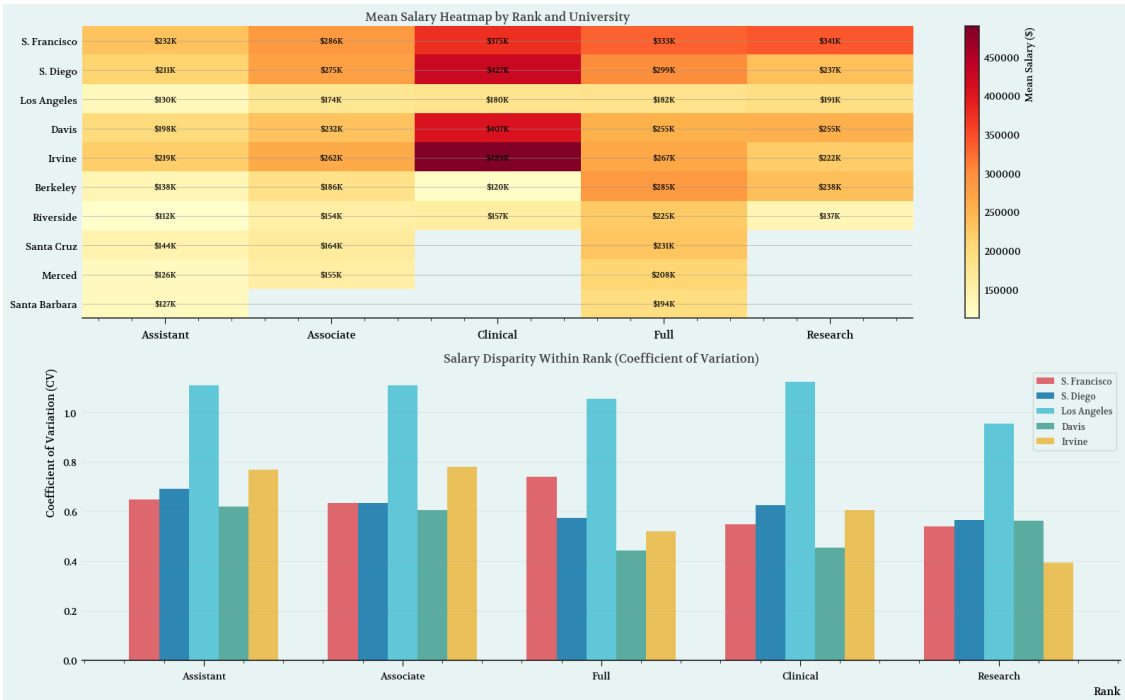
[105]:
```
fig, axes = plt.subplots(2, 2, figsize=(18, 12))

# 1. Professor Count by University
ax1 = axes[0, 0]
colors_univ = ['#005581' if i < 3 else '#3a7ca5' if i < 6 else '#6fb3d8' for i␣
↪in range(len(univ_counts))]
ax1.barh(range(len(univ_counts)), univ_counts['TotalProfessors'],␣
↪color=colors_univ)
ax1.set_yticks(range(len(univ_counts)))
ax1.set_yticklabels([name.replace('University of California, ', '').
↪replace('San ', 'S. ') for name in univ_counts['EmployerName']], fontsize=10)
ax1.invert_yaxis()
ax1.set_xlabel('Number of Professors')
ax1.set_title('Total Professors by University (2024)', fontweight='bold')
ax1.grid(axis='x', alpha=0.3)

# 2. Rank Distribution by University (Stacked Bar)
ax2 = axes[0, 1]
rank_colors = {'Assistant': '#005581', 'Associate': '#3a7ca5', 'Full':␣
↪'#e67e22', 'Clinical': '#c13d3d', 'Research': '#27ae60'}
rank_by_univ_sorted = rank_by_univ.loc[univ_counts['EmployerName']]
rank_by_univ_sorted.plot(kind='barh', stacked=True, ax=ax2, color=[rank_colors.
↪get(rank, '#999') for rank in rank_by_univ_sorted.columns])
ax2.set_xlabel('Number of Professors')
ax2.set_title('Rank Distribution by University', fontweight='bold')
ax2.set_yticklabels([name.replace('University of California, ', '').
↪replace('San ', 'S. ') for name in rank_by_univ_sorted.index], fontsize=10)
ax2.legend(title='Rank', loc='lower right', fontsize=9)
ax2.grid(axis='x', alpha=0.3)

# 3. Salary Range by University (Box plot)
ax3 = axes[1, 0]
box_data = [prof_2024[prof_2024['EmployerName'] == univ]['TotalPay'].values for␣
↪univ in univ_counts['EmployerName']]
bp = ax3.boxplot(box_data, labels=[name.replace('University of California, ',␣
↪'').replace('San ', 'S. ') for name in univ_counts['EmployerName']],␣
↪vert=True, patch_artist=True)
for patch in bp['boxes']:
```

```
    patch.set_facecolor('#005581')
    patch.set_alpha(0.7)
ax3.set_ylabel('Total Pay ($)')
ax3.set_title('Salary Distribution by University', fontweight='bold')
ax3.tick_params(axis='x', rotation=45)
ax3.grid(axis='y', alpha=0.3)

# 4. Compensation Range by Rank (showing variation across universities)
ax4 = axes[1, 1]
rank_order = ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']
x_pos = np.arange(len(rank_order))
ranges = [rank_ranges.loc[rank, 'Range'] if rank in rank_ranges.index else 0␣
 ↪for rank in rank_order]
range_pcts = [rank_ranges.loc[rank, 'Range %'] if rank in rank_ranges.index␣
 ↪else 0 for rank in rank_order]

bars = ax4.bar(x_pos, ranges, color=['#005581', '#3a7ca5', '#e67e22',␣
 ↪'#c13d3d', '#27ae60'])
ax4.set_xticks(x_pos)
ax4.set_xticklabels(rank_order, rotation=45, ha='right')
ax4.set_ylabel('Salary Range ($)')
ax4.set_title('Compensation Range by Rank Across Universities',␣
 ↪fontweight='bold')
ax4.grid(axis='y', alpha=0.3)

# Add percentage labels on bars
for i, (bar, pct) in enumerate(zip(bars, range_pcts)):
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height,
            f'{pct:.0f}%',
            ha='center', va='bottom', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()
```



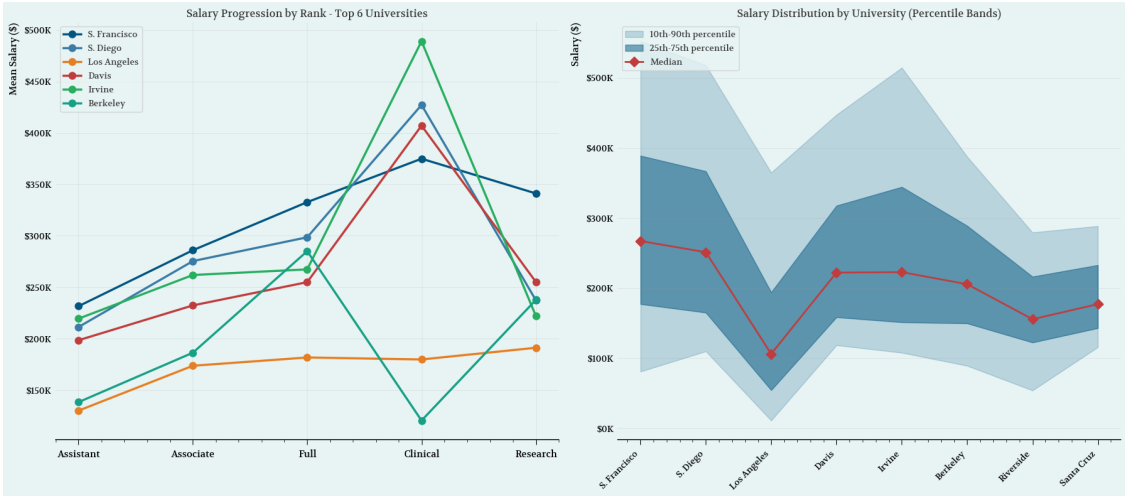
[106]:
```
# Salary heatmap by rank across universities
fig, axes = plt.subplots(2, 1, figsize=(16, 10))

# 1. Mean salary by rank and university (heatmap)
ax1 = axes[0]
salary_heatmap = prof_2024.pivot_table(values='TotalPay', index='EmployerName',␣
↪columns='Rank', aggfunc='mean')
salary_heatmap = salary_heatmap.loc[univ_counts['EmployerName']]  # Sort by␣
↪university size
im = ax1.imshow(salary_heatmap.values, cmap='YlOrRd', aspect='auto')
ax1.set_xticks(range(len(salary_heatmap.columns)))
ax1.set_yticks(range(len(salary_heatmap.index)))
ax1.set_xticklabels(salary_heatmap.columns, fontsize=11)
ax1.set_yticklabels([name.replace('University of California, ', '').
↪replace('San ', 'S. ') for name in salary_heatmap.index], fontsize=10)
ax1.set_title('Mean Salary Heatmap by Rank and University', fontweight='bold',␣
↪fontsize=12)

# Add colorbar
cbar = plt.colorbar(im, ax=ax1)
cbar.set_label('Mean Salary ($)', fontsize=10)

# Add salary values in cells
```

```
for i in range(len(salary_heatmap.index)):
    for j in range(len(salary_heatmap.columns)):
        val = salary_heatmap.values[i, j]
        if not np.isnan(val):
            text_color = 'white' if val > salary_heatmap.values.mean() else␣
 ↪'black'
            ax1.text(j, i, f'${val/1000:.0f}K', ha='center', va='center',␣
 ↪color=text_color, fontsize=8, fontweight='bold')

# 2. Coefficient of Variation (salary disparity within rank by university)
ax2 = axes[1]
cv_by_rank_univ = prof_2024.groupby(['Rank', 'EmployerName'])['TotalPay'].
 ↪apply(lambda x: x.std() / x.mean()).unstack()
cv_by_rank_univ = cv_by_rank_univ.loc[:, univ_counts['EmployerName']]

x_pos = np.arange(len(cv_by_rank_univ.index))
width = 0.15
rank_order_sorted = ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']
colors_cv = ['#005581', '#3a7ca5', '#e67e22', '#c13d3d', '#27ae60']

for i, univ in enumerate(cv_by_rank_univ.columns):
    if i < 5:  # Plot top 5 universities only for clarity
        cv_values = [cv_by_rank_univ.loc[rank, univ] if rank in cv_by_rank_univ.
 ↪index else 0 for rank in rank_order_sorted]
        ax2.bar(x_pos + i*width, cv_values, width, label=univ.
 ↪replace('University of California, ', '').replace('San ', 'S. '), alpha=0.8)

ax2.set_xlabel('Rank', fontsize=11)
ax2.set_ylabel('Coefficient of Variation (CV)', fontsize=11)
ax2.set_title('Salary Disparity Within Rank (Coefficient of Variation)',␣
 ↪fontweight='bold', fontsize=12)
ax2.set_xticks(x_pos + width * 2)
ax2.set_xticklabels(rank_order_sorted, fontsize=10)
ax2.legend(loc='upper right', fontsize=9)
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



[107]:
```
# Distribution comparison: median vs mean salary by university and rank
fig, axes = plt.subplots(1, 2, figsize=(18, 8))

# 1. Mean salary progression by rank across top universities
ax1 = axes[0]
rank_order_sorted = ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']
top_univs = univ_counts['EmployerName'].head(6)
colors_line = ['#005581', '#3a7ca5', '#e67e22', '#c13d3d', '#27ae60', '#16a085']

for idx, univ in enumerate(top_univs):
    univ_data = prof_2024[prof_2024['EmployerName'] == univ]
    means = [univ_data[univ_data['Rank'] == rank]['TotalPay'].mean() if rank in␣
 ↪univ_data['Rank'].values else None for rank in rank_order_sorted]
    ax1.plot(range(len(rank_order_sorted)), means, marker='o', linewidth=2.5,␣
 ↪markersize=8, label=univ.replace('University of California, ', '').
 ↪replace('San ', 'S. '), color=colors_line[idx])

ax1.set_xticks(range(len(rank_order_sorted)))
ax1.set_xticklabels(rank_order_sorted, fontsize=11)
ax1.set_ylabel('Mean Salary ($)', fontsize=11)
ax1.set_title('Salary Progression by Rank - Top 6 Universities',␣
 ↪fontweight='bold', fontsize=12)
ax1.legend(loc='upper left', fontsize=10)
ax1.grid(True, alpha=0.3)
```

```python
ax1.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x/1000:.0f}K'))

# 2. Salary percentile bands by university (quartiles)
ax2 = axes[1]
univs_to_plot = univ_counts['EmployerName'].head(8)
univ_labels = [name.replace('University of California, ', '').replace('San ',␣
↪'S. ') for name in univs_to_plot]

percentiles_data = []
for univ in univs_to_plot:
    univ_data = prof_2024[prof_2024['EmployerName'] == univ]['TotalPay']
    percentiles_data.append([
        univ_data.quantile(0.1),
        univ_data.quantile(0.25),
        univ_data.quantile(0.50),
        univ_data.quantile(0.75),
        univ_data.quantile(0.90)
    ])

percentiles_data = np.array(percentiles_data)
x_pos = np.arange(len(univs_to_plot))

# Create violin-like visualization using percentile bands
ax2.fill_between(x_pos - 0.3, percentiles_data[:, 0], percentiles_data[:, 4],␣
↪alpha=0.2, color='#005581', label='10th-90th percentile')
ax2.fill_between(x_pos - 0.3, percentiles_data[:, 1], percentiles_data[:, 3],␣
↪alpha=0.5, color='#005581', label='25th-75th percentile')
ax2.plot(x_pos - 0.3, percentiles_data[:, 2], marker='D', color='#c13d3d',␣
↪linewidth=2, markersize=8, label='Median')

ax2.set_xticks(x_pos - 0.3)
ax2.set_xticklabels(univ_labels, rotation=45, ha='right', fontsize=10)
ax2.set_ylabel('Salary ($)', fontsize=11)
ax2.set_title('Salary Distribution by University (Percentile Bands)',␣
↪fontweight='bold', fontsize=12)
ax2.legend(loc='upper left', fontsize=10)
ax2.grid(axis='y', alpha=0.3)
ax2.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x/1000:.0f}K'))

plt.tight_layout()
plt.show()
```



3 Analysis: 2024 Snapshot

Focused analysis on the 2024 professor subset.

[94]:
```
# Create subset with positions that have "Prof " (with space) or "Prof-" to␣
↪exclude "Professional"
prof_subset_24 = df24[df24['Position'].str.contains(r'Prof[\s\-]', case=False,␣
↪na=False, regex=True)]
```

[95]:
```
# Salary Analysis for Professor Positions in 2024
print("=" * 70)
print("PROFESSOR SALARY ANALYSIS - 2024 DATA")
print("=" * 70)

print(f"\nUsing column: '{salary_col}'")
print(f"Total professors: {len(prof_subset_24)}")
print(f"Non-zero {pay_col} records: {(prof_subset_24[pay_col] > 0).sum()}")

print("\n1. SALARY STATISTICS")
print("-" * 70)
print(prof_subset_24[salary_col].describe())

# Group by position type
print("\n2. SALARY BY POSITION TYPE (Top 15)")
print("-" * 70)
position_stats = prof_subset_24.groupby('Position')[salary_col].agg(['count',␣
↪'mean', 'median', 'min', 'max'])
position_stats = position_stats.sort_values('mean', ascending=False)
print(position_stats.head(15))
```

```python
# Check for salary by campus/organization
print(f"\n3. SALARY BY EMPLOYER")
print("-" * 70)
org_stats = prof_subset_24.groupby('EmployerName')[salary_col].agg(['count',␣
↪'mean', 'median'])
org_stats = org_stats.sort_values('mean', ascending=False)
print(org_stats.head(10))

# Visualize salary distribution
print("\n4. SALARY DISTRIBUTION VISUALIZATION")
print("-" * 70)
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Histogram
prof_subset_24[salary_col].hist(bins=50, ax=axes[0, 0], edgecolor='black',␣
↪color='steelblue')
axes[0, 0].set_title(f'{salary_label} Distribution (Histogram)', fontsize=12,␣
↪fontweight='bold')
axes[0, 0].set_xlabel(salary_label)
axes[0, 0].set_ylabel('Frequency')

# Box plot by top 20 positions
top_positions = prof_subset_24['Position'].value_counts().head(20).index
prof_subset_top = prof_subset_24[prof_subset_24['Position'].isin(top_positions)]
prof_subset_top.boxplot(column=salary_col, by='Position', ax=axes[0, 1])
axes[0, 1].set_title(f'{salary_label} by Top 20 Position Types', fontsize=12,␣
↪fontweight='bold')
axes[0, 1].set_xlabel('Position')
axes[0, 1].set_ylabel(salary_label)
plt.setp(axes[0, 1].xaxis.get_majorticklabels(), rotation=45, ha='right',␣
↪fontsize=8)

# Top 10 highest paid
top_10 = prof_subset_24.nlargest(10, salary_col)[[salary_col, 'Position',␣
↪'EmployerName']]
axes[1, 0].barh(range(len(top_10)), top_10[salary_col].values,␣
↪color='darkgreen')
axes[1, 0].set_yticks(range(len(top_10)))
axes[1, 0].set_yticklabels([f"{pos[:30]}" for pos in top_10['Position'].
↪values], fontsize=9)
axes[1, 0].set_title(f'Top 10 Highest {salary_label}', fontsize=12,␣
↪fontweight='bold')
axes[1, 0].set_xlabel(salary_label)
axes[1, 0].ticklabel_format(style='plain', axis='x')

# Salary percentiles
```

```
percentiles = [10, 25, 50, 75, 90, 95, 99]
percentile_values = [np.percentile(prof_subset_24[salary_col].dropna(), p) for␣
 ↪p in percentiles]
axes[1, 1].plot(percentiles, percentile_values, marker='o', linewidth=2,␣
 ↪markersize=8, color='darkred')
axes[1, 1].set_title(f'{salary_label} Percentiles', fontsize=12,␣
 ↪fontweight='bold')
axes[1, 1].set_xlabel('Percentile')
axes[1, 1].set_ylabel(salary_label)
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].ticklabel_format(style='plain', axis='y')

plt.tight_layout()
plt.show()
```

```
======================================================================
PROFESSOR SALARY ANALYSIS - 2024 DATA
======================================================================

Using column: 'RealTotalPay'
Total professors: 18898
Non-zero TotalPay records: 18898

1. SALARY STATISTICS
----------------------------------------------------------------------
count    1.889800e+04
mean     2.472760e+05
std      1.798689e+05
min      2.900000e+01
25%      1.341338e+05
50%      2.086905e+05
75%      3.198462e+05
max      2.009134e+06
Name: RealTotalPay, dtype: float64

2. SALARY BY POSITION TYPE (Top 15)
----------------------------------------------------------------------
                                count           mean    median       min  \
Position
Prof Of Clin-Hcomp                833  448421.887155  414648.0    1875.0
Associate Prof Of Clin-Hcomp      147  424830.428571  388841.0    5610.0
Associatec Prof Of Clin-Hcomp     217  401425.133641  365721.0     262.0
Act Prof-Ay-1/9                     1  362951.000000  362951.0  362951.0
Hs Clin Prof-Hcomp               1538  357210.493498  329499.0     470.0
Prof-Hcomp                       1078  356497.203154  307883.5     428.0
Assistant Prof Of Clin-Hcomp      216  324727.606481  314310.0     305.0
Prof-Ay-Law                       155  318521.464516  325354.0      40.0
```

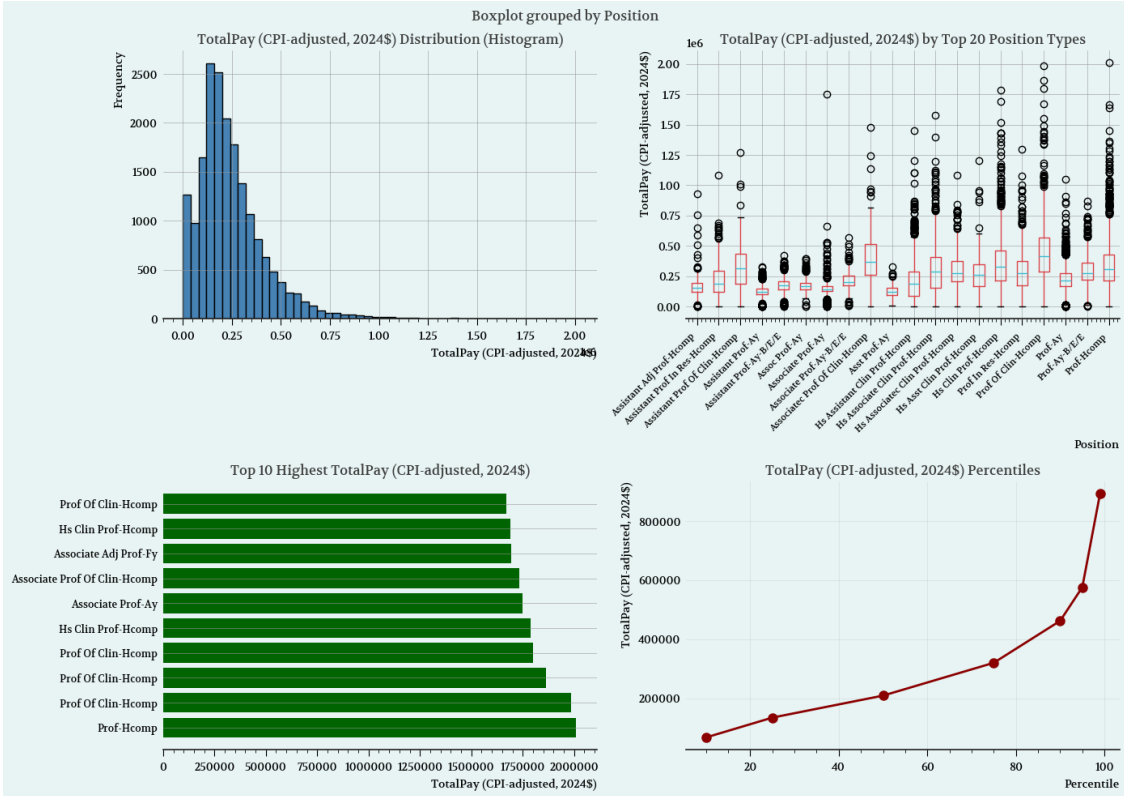
Prof Of Teach-Ay-Law	28	318034.464286	333003.5	119731.0
Asst Prof Of Clin-Hcomp	132	316500.825758	312372.0	35417.0
Hs Associatec Clin Prof-Hcomp	422	307370.500000	277878.5	465.0
Hs Associate Clin Prof-Hcomp	1053	306373.830959	288999.0	450.0
Vis Assistant Prof-Hcomp	11	301939.272727	278755.0	5792.0
Prof-Fy-B/E/E	14	299602.857143	299962.5	199721.0
Prof In Res-Hcomp	572	297256.220280	274618.0	583.0

Position	max
Prof Of Clin-Hcomp	1983544.0
Associate Prof Of Clin-Hcomp	1735259.0
Associatec Prof Of Clin-Hcomp	1476706.0
Act Prof-Ay-1/9	362951.0
Hs Clin Prof-Hcomp	1787618.0
Prof-Hcomp	2009134.0
Assistant Prof Of Clin-Hcomp	1268041.0
Prof-Ay-Law	677221.0
Prof Of Teach-Ay-Law	444066.0
Asst Prof Of Clin-Hcomp	1260373.0
Hs Associatec Clin Prof-Hcomp	1083939.0
Hs Associate Clin Prof-Hcomp	1574394.0
Vis Assistant Prof-Hcomp	563636.0
Prof-Fy-B/E/E	410115.0
Prof In Res-Hcomp	1294299.0

3. SALARY BY EMPLOYER

EmployerName	count	mean	median
University of California, San Francisco	3948	298739.708207	267289.5
University of California, San Diego	3259	292982.705738	251350.0
University of California, Irvine	2274	277151.473615	222895.5
University of California, Davis	2730	256476.404029	222273.0
University of California, Berkeley	1690	224817.206509	205692.0
University of California, Santa Cruz	691	190954.746744	177257.0
University of California, Riverside	986	167993.279919	155699.5
University of California, Merced	337	164933.498516	154633.0
University of California, Santa Barbara	54	164362.574074	162771.0
University of California, Los Angeles	2929	159219.451349	106182.0

4. SALARY DISTRIBUTION VISUALIZATION



4 Analysis: Multi-year Trends

Professor salary trends across 2013–2024.

[96]:
```
# Collect professor salary data across all years (refined regex: "Prof " or
↪"Prof-" only)
print("=" * 70)
print("PROFESSOR SALARY TRENDS ACROSS ALL YEARS (2013-2024)")
print("=" * 70)

# Create professor subsets for all years using refined pattern
prof_subsets = {}
years = list(range(13, 25))  # 13-24 for 2013-2024

for year in years:
    df_name = f'df{year:02d}'
    if df_name in globals():
        df = globals()[df_name]
        # Use regex to match "Prof " (with space) or "Prof-" to exclude
↪"Professional"
```

```
        prof_subset = df[df['Position'].str.contains(r'Prof[\s\-]', case=False,␣
↪na=False, regex=True)]
        prof_subsets[f'20{year}'] = prof_subset
        print(f"{df_name} -> 20{year}: {len(prof_subset)} professors")

print("\n" + "=" * 70)
print("YEAR-BY-YEAR STATISTICS")
print("=" * 70)

# Collect statistics for each year
yearly_stats = []
for year_str in sorted(prof_subsets.keys()):
    df_prof = prof_subsets[year_str]
    stats = {
        'Year': year_str,
        'Count': len(df_prof),
        'Mean': df_prof[salary_col].mean(),
        'Median': df_prof[salary_col].median(),
        'Std': df_prof[salary_col].std(),
        'Min': df_prof[salary_col].min(),
        'Max': df_prof[salary_col].max(),
        'Q25': df_prof[salary_col].quantile(0.25),
        'Q75': df_prof[salary_col].quantile(0.75),
    }
    yearly_stats.append(stats)

yearly_df = pd.DataFrame(yearly_stats)
print(yearly_df.to_string(index=False))

print("\n" + "=" * 70)
print("YEAR-OVER-YEAR CHANGE IN MEAN SALARY")
print("=" * 70)
yearly_df['YoY_Change'] = yearly_df['Mean'].diff()
yearly_df['YoY_Change_Pct'] = yearly_df['Mean'].pct_change() * 100
print(yearly_df[['Year', 'Mean', 'YoY_Change', 'YoY_Change_Pct']].
↪to_string(index=False))

print("\n" + "=" * 70)
print("ANALYSIS: Changes Starting in 2023")
print("=" * 70)

mean_2022 = yearly_df[yearly_df['Year'] == '2022']['Mean'].values[0]
mean_2023 = yearly_df[yearly_df['Year'] == '2023']['Mean'].values[0]
mean_2024 = yearly_df[yearly_df['Year'] == '2024']['Mean'].values[0]

change_2022_to_2023 = mean_2023 - mean_2022
change_2023_to_2024 = mean_2024 - mean_2023
```

```
pct_change_2022_to_2023 = (change_2022_to_2023 / mean_2022) * 100
pct_change_2023_to_2024 = (change_2023_to_2024 / mean_2023) * 100

print(f"\n2022 Mean Salary: ${mean_2022:,.2f}")
print(f"2023 Mean Salary: ${mean_2023:,.2f}")
print(f"2024 Mean Salary: ${mean_2024:,.2f}")
print(f"\n2022 → 2023 Change: ${change_2022_to_2023:,.2f}␣
↪({pct_change_2022_to_2023:+.2f}%)")
print(f"2023 → 2024 Change: ${change_2023_to_2024:,.2f}␣
↪({pct_change_2023_to_2024:+.2f}%)")

# Check if 2023 represents a significant deviation
avg_pre_2023 = yearly_df[yearly_df['Year'] < '2023']['Mean'].mean()
std_pre_2023 = yearly_df[yearly_df['Year'] < '2023']['Mean'].std()

print(f"\nAverage Mean Salary (2013-2022): ${avg_pre_2023:,.2f}")
print(f"Std Dev (2013-2022): ${std_pre_2023:,.2f}")
print(f"2023 Deviation from Historical Mean: ${mean_2023 - avg_pre_2023:,.2f}␣
↪({((mean_2023 - avg_pre_2023) / std_pre_2023):.2f} std devs)")
print(f"2024 Deviation from Historical Mean: ${mean_2024 - avg_pre_2023:,.2f}␣
↪({((mean_2024 - avg_pre_2023) / std_pre_2023):.2f} std devs)")
```

```
======================================================================
PROFESSOR SALARY TRENDS ACROSS ALL YEARS (2013-2024)
======================================================================
df13 -> 2013: 15711 professors
df14 -> 2014: 16042 professors
df15 -> 2015: 16508 professors
df16 -> 2016: 17176 professors
df17 -> 2017: 17739 professors
df18 -> 2018: 18151 professors
df19 -> 2019: 17963 professors
df20 -> 2020: 19224 professors
df21 -> 2021: 19415 professors
df22 -> 2022: 19737 professors
df23 -> 2023: 20176 professors
df24 -> 2024: 18898 professors

======================================================================
YEAR-BY-YEAR STATISTICS
======================================================================
Year  Count          Mean        Median           Std        Min          Max
Q25           Q75
2013  15711 232764.653343 197301.774404 165481.704637 134.672383 3.006135e+06
132650.276960 292834.322582
2014  16042 237534.450692 201478.367951 169705.497413   1.325516 3.053096e+06
134383.089013 299160.581623
```

```
2015  16508 247859.642434 210133.282857 177385.097473  21.187062 3.092975e+06
139798.857325 312564.783426
2016  17176 255367.132757 215845.156916 185926.241690  10.457583 3.244736e+06
142830.969325 322899.429225
2017  17739 259631.785831 219255.867113 189629.446711   2.560623 3.177977e+06
144550.344152 329699.999110
2018  18151 263062.599059 223575.607360 189170.589085   1.250304 3.104816e+06
147973.476409 331593.744531
2019  17963 268525.394471 227277.752542 192995.594860  18.422947 3.048689e+06
151170.103302 339700.100232
2020  19224 273766.569592 236424.784986 186746.171257   2.424870 3.106864e+06
157762.015499 349233.961816
2021  19415 274668.661334 233570.294280 192808.555380 115.800840 3.925512e+06
155826.821342 352060.608789
2022  19737 267489.864198 226288.780350 187107.272435  35.383590 3.674256e+06
150550.742070 342844.470270
2023  20176 272178.970925 231371.990000 191667.866692   3.090000 3.478839e+06
152897.577500 345164.587500
2024  18898 247276.016721 208690.500000 179868.860124  29.000000 2.009134e+06
134133.750000 319846.250000

======================================================================
YEAR-OVER-YEAR CHANGE IN MEAN SALARY
======================================================================
Year          Mean    YoY_Change  YoY_Change_Pct
2013 232764.653343           NaN             NaN
2014 237534.450692   4769.797349        2.049193
2015 247859.642434  10325.191742        4.346819
2016 255367.132757   7507.490323        3.028928
2017 259631.785831   4264.653074        1.670009
2018 263062.599059   3430.813229        1.321415
2019 268525.394471   5462.795411        2.076614
2020 273766.569592   5241.175121        1.951836
2021 274668.661334    902.091742        0.329511
2022 267489.864198  -7178.797136       -2.613621
2023 272178.970925   4689.106728        1.753004
2024 247276.016721 -24902.954204       -9.149478

======================================================================
ANALYSIS: Changes Starting in 2023
======================================================================

2022 Mean Salary: $267,489.86
2023 Mean Salary: $272,178.97
2024 Mean Salary: $247,276.02

2022 → 2023 Change: $4,689.11 (+1.75%)
2023 → 2024 Change: $-24,902.95 (-9.15%)
```

```
Average Mean Salary (2013-2022): $258,067.08
Std Dev (2013-2022): $14,609.50
2023 Deviation from Historical Mean: $14,111.90 (0.97 std devs)
2024 Deviation from Historical Mean: $-10,791.06 (-0.74 std devs)
```

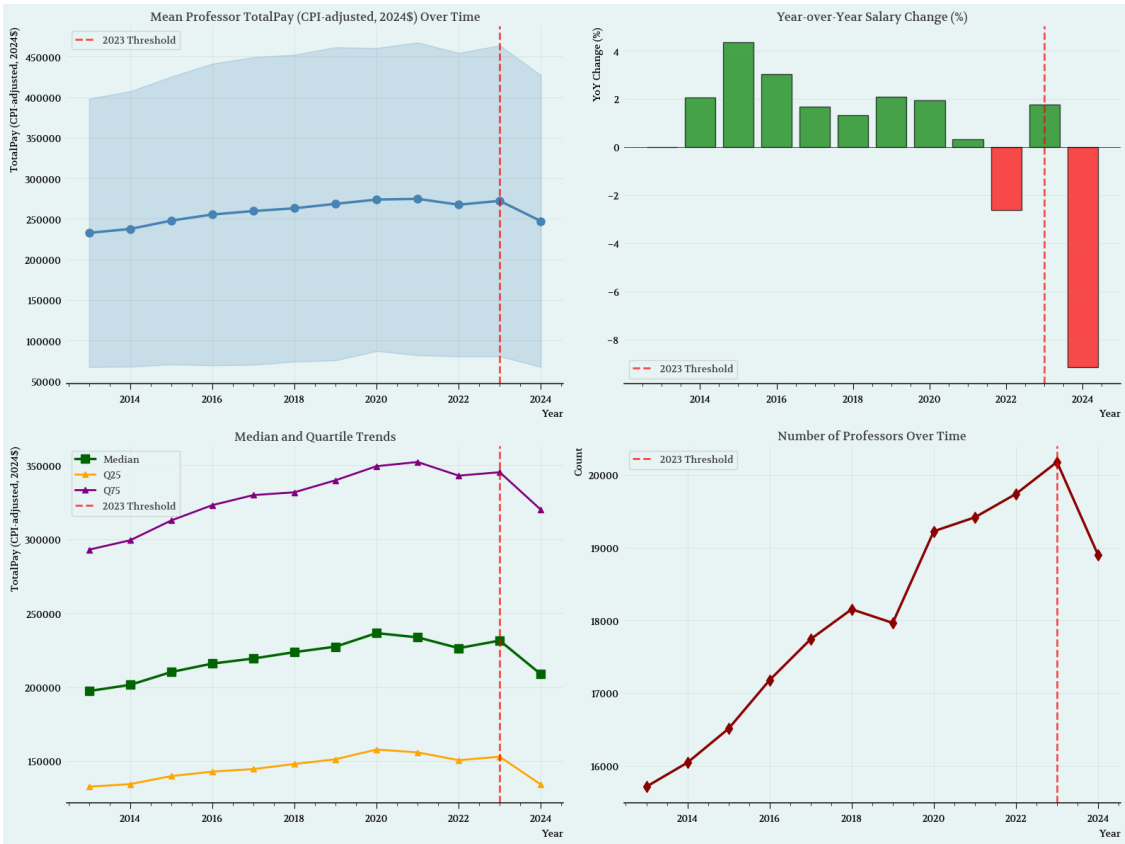
5 Visualization: Trends

Year-by-year trend charts for professor compensation.

[97]:
```
# Visualize salary trends across years
print("\n" + "=" * 70)
print("SALARY TREND VISUALIZATION")
print("=" * 70)

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Extract years for plotting
years_list = yearly_df['Year'].tolist()
years_numeric = [int(y) for y in years_list]

# 1. Mean salary trend
axes[0, 0].plot(years_numeric, yearly_df['Mean'], marker='o', linewidth=2.5,␣
↪markersize=8, color='steelblue')
axes[0, 0].axvline(x=2023, color='red', linestyle='--', linewidth=2, alpha=0.7,␣
↪label='2023 Threshold')
axes[0, 0].fill_between(years_numeric, yearly_df['Mean'] - yearly_df['Std'],␣
↪yearly_df['Mean'] + yearly_df['Std'], alpha=0.2, color='steelblue')
axes[0, 0].set_title(f'Mean Professor {salary_label} Over Time', fontsize=12,␣
↪fontweight='bold')
axes[0, 0].set_xlabel('Year')
axes[0, 0].set_ylabel(salary_label)
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].legend()
axes[0, 0].ticklabel_format(style='plain', axis='y')

# 2. Year-over-year percentage change
yoy_change_pct = yearly_df['YoY_Change_Pct'].fillna(0)
colors = ['green' if x > 0 else 'red' for x in yoy_change_pct]
axes[0, 1].bar(years_numeric, yoy_change_pct, color=colors, alpha=0.7,␣
↪edgecolor='black')
axes[0, 1].axvline(x=2023, color='red', linestyle='--', linewidth=2, alpha=0.7,␣
↪label='2023 Threshold')
axes[0, 1].axhline(y=0, color='black', linestyle='-', linewidth=0.5)
axes[0, 1].set_title('Year-over-Year Salary Change (%)', fontsize=12,␣
↪fontweight='bold')
axes[0, 1].set_xlabel('Year')
```

```
axes[0, 1].set_ylabel('YoY Change (%)')
axes[0, 1].grid(True, alpha=0.3, axis='y')
axes[0, 1].legend()

# 3. Median + Quartiles
axes[1, 0].plot(years_numeric, yearly_df['Median'], marker='s', linewidth=2.5,␣
 ↪markersize=8, color='darkgreen', label='Median')
axes[1, 0].plot(years_numeric, yearly_df['Q25'], marker='^', linewidth=2,␣
 ↪markersize=6, color='orange', label='Q25')
axes[1, 0].plot(years_numeric, yearly_df['Q75'], marker='^', linewidth=2,␣
 ↪markersize=6, color='purple', label='Q75')
axes[1, 0].axvline(x=2023, color='red', linestyle='--', linewidth=2, alpha=0.7,␣
 ↪label='2023 Threshold')
axes[1, 0].set_title('Median and Quartile Trends', fontsize=12,␣
 ↪fontweight='bold')
axes[1, 0].set_xlabel('Year')
axes[1, 0].set_ylabel(salary_label)
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].legend()
axes[1, 0].ticklabel_format(style='plain', axis='y')

# 4. Count of professors over time
axes[1, 1].plot(years_numeric, yearly_df['Count'], marker='d', linewidth=2.5,␣
 ↪markersize=8, color='darkred')
axes[1, 1].axvline(x=2023, color='red', linestyle='--', linewidth=2, alpha=0.7,␣
 ↪label='2023 Threshold')
axes[1, 1].set_title('Number of Professors Over Time', fontsize=12,␣
 ↪fontweight='bold')
axes[1, 1].set_xlabel('Year')
axes[1, 1].set_ylabel('Count')
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].legend()

plt.tight_layout()
plt.show()

print("\n  Visualizations complete!")
```

```
======================================================================
SALARY TREND VISUALIZATION
======================================================================
```




```
Visualizations complete!
```

6 Analysis: Rank Variation

Variation and dispersion metrics by professor rank.

[98]:
```
# Categorize professor positions into ranks
def categorize_rank(position):
    """Categorize position into professor rank"""
    pos_lower = position.lower()

    if 'asst' in pos_lower or 'assistant' in pos_lower:
        return 'Assistant'
    elif 'assoc' in pos_lower:
        return 'Associate'
    elif 'clin' in pos_lower:
        return 'Clinical'
    elif 'res' in pos_lower or 'research' in pos_lower:
        return 'Research'
    elif 'in res' in pos_lower:
```

```
        return 'Research'
    else:
        return 'Full'

# Add rank category to each professor subset
for year_str in prof_subsets.keys():
    prof_subsets[year_str]['Rank'] = prof_subsets[year_str]['Position'].
 ↪apply(categorize_rank)

print("=" * 80)
print("SALARY VARIATION WITHIN PROFESSOR RANKS BY YEAR")
print("=" * 80)

# Analyze variation metrics by rank and year
variation_analysis = []

for year_str in sorted(prof_subsets.keys()):
    df_year = prof_subsets[year_str]
    year_int = int(year_str)

    for rank in ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']:
        rank_data = df_year[df_year['Rank'] == rank][salary_col]

        if len(rank_data) > 0:
            mean_sal = rank_data.mean()
            std_sal = rank_data.std()
            cv = (std_sal / mean_sal * 100) if mean_sal > 0 else 0  #␣
 ↪Coefficient of Variation
            min_sal = rank_data.min()
            max_sal = rank_data.max()
            range_sal = max_sal - min_sal
            q25 = rank_data.quantile(0.25)
            q75 = rank_data.quantile(0.75)
            iqr = q75 - q25
            count = len(rank_data)

            variation_analysis.append({
                'Year': year_int,
                'Rank': rank,
                'Count': count,
                'Mean': mean_sal,
                'Std': std_sal,
                'CV%': cv,
                'Min': min_sal,
                'Max': max_sal,
                'Range': range_sal,
                'Q25': q25,
```

```
                'Q75': q75,
                'IQR': iqr,
            })

variation_df = pd.DataFrame(variation_analysis)

# Display variation metrics for each rank
for rank in ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']:
    rank_data = variation_df[variation_df['Rank'] == rank]
    if len(rank_data) > 0:
        print(f"\n{rank.upper()} PROFESSORS")
        print("-" * 80)
        print(rank_data[['Year', 'Count', 'Mean', 'Std', 'CV%', 'IQR']].
 ↪to_string(index=False))

        # Calculate trends
        years_in_data = sorted(rank_data['Year'].unique())
        if len(years_in_data) > 1:
            cv_2013_2022 = rank_data[(rank_data['Year'] >= 2013) &␣
 ↪(rank_data['Year'] < 2023)]['CV%'].mean()
            cv_2023_2024 = rank_data[(rank_data['Year'] >= 2023)]['CV%'].mean()

            std_change = rank_data[rank_data['Year'] == 2024]['Std'].values
            std_2022 = rank_data[rank_data['Year'] == 2022]['Std'].values

            if len(std_change) > 0 and len(std_2022) > 0:
                std_change_pct = ((std_change[0] - std_2022[0]) / std_2022[0] *␣
 ↪100) if std_2022[0] > 0 else 0
                print(f"\n  Avg CV (2013-2022): {cv_2013_2022:.2f}%")
                print(f"  Avg CV (2023-2024): {cv_2023_2024:.2f}%")
                print(f"  Std Dev Change (2022→2024): {std_change_pct:+.2f}%")
```

```
================================================================================
SALARY VARIATION WITHIN PROFESSOR RANKS BY YEAR
================================================================================

ASSISTANT PROFESSORS
--------------------------------------------------------------------------------
 Year  Count          Mean           Std       CV%           IQR
 2013   4166 181522.112070 134612.262501 74.157501 135716.093755
 2014   4078 180795.281269 137333.945566 75.961023 142374.953815
 2015   4333 188376.564101 141097.920441 74.902056 149416.459370
 2016   4744 192849.079810 147869.120762 76.676083 154817.319695
 2017   5070 196829.782228 148749.203508 75.572508 157475.726667
 2018   5321 198311.230073 149661.374805 75.467927 155764.120529
 2019   5607 209246.709368 153629.257004 73.420154 158900.985839
 2020   6045 213159.881347 147904.155423 69.386488 160531.216571
```

2021	5540	217135.234144	154082.173093	70.961387	169436.025561
2022	5500	209326.040730	155365.812981	74.221923	165836.989065
2023	5536	212355.568486	159968.298467	75.330400	169478.775000
2024	5514	183208.623685	143030.399523	78.069687	149937.250000

Avg CV (2013-2022): 74.07%
Avg CV (2023-2024): 76.70%
Std Dev Change (2022→2024): -7.94%

ASSOCIATE PROFESSORS

Year	Count	Mean	Std	CV%	IQR
2013	3487	202378.641299	152872.274518	75.537751	119102.908614
2014	3608	207477.042707	159065.079516	76.666352	124722.068718
2015	3651	216354.937153	167665.253162	77.495460	135091.356943
2016	3764	227722.484419	179705.256358	78.914147	145344.383925
2017	3861	236726.616135	189357.283332	79.989858	155821.564495
2018	3871	244169.477961	192382.229273	78.790450	161978.756477
2019	3762	251727.419385	201723.516823	80.135695	167286.190033
2020	4002	253367.926245	186970.174766	73.793940	163341.943534
2021	4365	250494.216165	187775.462931	74.961995	171195.329822
2022	4567	245241.694148	180051.401114	73.417941	172496.609595
2023	4811	248390.806743	176235.843628	70.951033	174763.705000
2024	4652	233612.627472	168535.788767	72.143270	158023.000000

Avg CV (2013-2022): 76.97%
Avg CV (2023-2024): 71.55%
Std Dev Change (2022→2024): -6.40%

FULL PROFESSORS

Year	Count	Mean	Std	CV%	IQR
2013	6161	253735.038983	153741.331686	60.591289	129393.225383
2014	6298	257984.609838	152619.206704	59.158260	132465.730728
2015	6388	268017.111225	158798.118584	59.249246	138899.400325
2016	6459	277336.061222	164771.141058	59.412087	139107.416347
2017	6533	280120.364914	167023.897555	59.625760	143123.437232
2018	6599	283798.763430	164172.899985	57.848349	144942.739554
2019	6287	285303.014199	163479.732229	57.300387	141751.064749
2020	6711	297843.857510	166536.592598	55.914060	149047.034194
2021	6756	287675.363346	163940.975880	56.988188	148882.534469
2022	6760	276803.112114	154330.091307	55.754464	143345.356470
2023	6730	280647.634293	157327.726199	56.058811	143732.380000
2024	5607	261328.559836	153136.369118	58.599171	140714.500000

Avg CV (2013-2022): 58.18%
Avg CV (2023-2024): 57.33%
Std Dev Change (2022→2024): -0.77%

CLINICAL PROFESSORS

Year	Count	Mean	Std	CV%	IQR
2013	1324	350712.793956	228760.179629	65.227213	241035.283990
2014	1476	358723.185427	236980.701705	66.062276	237470.423836
2015	1576	378119.352777	242497.367019	64.132493	254181.847258
2016	1648	391538.915926	256751.874066	65.575059	266981.755065
2017	1711	397143.457317	260869.907212	65.686568	267739.974929
2018	1792	401294.800005	257341.440031	64.127778	262985.814062
2019	1766	413843.528137	265633.113775	64.186847	264039.821548
2020	1890	408756.053989	253595.536407	62.040803	251701.463400
2021	2154	415485.396567	268798.214797	64.694985	261055.913156
2022	2308	408832.216171	256462.779877	62.730570	259102.771155
2023	2466	414806.651010	269361.876714	64.936730	262226.670000
2024	2524	371263.788431	243612.721270	65.617151	270584.000000

Avg CV (2013-2022): 64.45%
Avg CV (2023-2024): 65.28%
Std Dev Change (2022→2024): -5.01%

RESEARCH PROFESSORS

Year	Count	Mean	Std	CV%	IQR
2013	573	292224.630386	192149.110267	65.753907	161872.163940
2014	582	292791.815201	193865.068047	66.212598	164755.952981
2015	560	316982.219684	215267.008780	67.911383	175562.949359
2016	561	316564.444530	221142.241140	69.856942	170003.690319
2017	564	326491.114545	227151.387271	69.573528	175063.042626
2018	568	321385.228230	222100.988702	69.107404	178052.039333
2019	541	330368.001415	232720.400302	70.442779	188860.995872
2020	576	328089.479985	215979.385579	65.829415	188843.084573
2021	600	329775.516857	233519.931572	70.811785	190723.693978
2022	602	321198.697124	216452.637153	67.389015	172284.039997
2023	633	330493.742227	215626.859412	65.243855	177604.960000
2024	601	289026.344426	186296.716024	64.456656	208552.000000

Avg CV (2013-2022): 68.29%
Avg CV (2023-2024): 64.85%
Std Dev Change (2022→2024): -13.93%

ASSISTANT PROFESSORS

Year	Count	Mean	Std	CV%	IQR
2013	4166	181522.112070	134612.262501	74.157501	135716.093755
2014	4078	180795.281269	137333.945566	75.961023	142374.953815
2015	4333	188376.564101	141097.920441	74.902056	149416.459370
2016	4744	192849.079810	147869.120762	76.676083	154817.319695

2017	5070	196829.782228	148749.203508	75.572508	157475.726667
2018	5321	198311.230073	149661.374805	75.467927	155764.120529
2019	5607	209246.709368	153629.257004	73.420154	158900.985839
2020	6045	213159.881347	147904.155423	69.386488	160531.216571
2021	5540	217135.234144	154082.173093	70.961387	169436.025561
2022	5500	209326.040730	155365.812981	74.221923	165836.989065
2023	5536	212355.568486	159968.298467	75.330400	169478.775000
2024	5514	183208.623685	143030.399523	78.069687	149937.250000

Avg CV (2013-2022): 74.07%
Avg CV (2023-2024): 76.70%
Std Dev Change (2022→2024): -7.94%

ASSOCIATE PROFESSORS

Year	Count	Mean	Std	CV%	IQR
2013	3487	202378.641299	152872.274518	75.537751	119102.908614
2014	3608	207477.042707	159065.079516	76.666352	124722.068718
2015	3651	216354.937153	167665.253162	77.495460	135091.356943
2016	3764	227722.484419	179705.256358	78.914147	145344.383925
2017	3861	236726.616135	189357.283332	79.989858	155821.564495
2018	3871	244169.477961	192382.229273	78.790450	161978.756477
2019	3762	251727.419385	201723.516823	80.135695	167286.190033
2020	4002	253367.926245	186970.174766	73.793940	163341.943534
2021	4365	250494.216165	187775.462931	74.961995	171195.329822
2022	4567	245241.694148	180051.401114	73.417941	172496.609595
2023	4811	248390.806743	176235.843628	70.951033	174763.705000
2024	4652	233612.627472	168535.788767	72.143270	158023.000000

Avg CV (2013-2022): 76.97%
Avg CV (2023-2024): 71.55%
Std Dev Change (2022→2024): -6.40%

FULL PROFESSORS

Year	Count	Mean	Std	CV%	IQR
2013	6161	253735.038983	153741.331686	60.591289	129393.225383
2014	6298	257984.609838	152619.206704	59.158260	132465.730728
2015	6388	268017.111225	158798.118584	59.249246	138899.400325
2016	6459	277336.061222	164771.141058	59.412087	139107.416347
2017	6533	280120.364914	167023.897555	59.625760	143123.437232
2018	6599	283798.763430	164172.899985	57.848349	144942.739554
2019	6287	285303.014199	163479.732229	57.300387	141751.064749
2020	6711	297843.857510	166536.592598	55.914060	149047.034194
2021	6756	287675.363346	163940.975880	56.988188	148882.534469
2022	6760	276803.112114	154330.091307	55.754464	143345.356470
2023	6730	280647.634293	157327.726199	56.058811	143732.380000
2024	5607	261328.559836	153136.369118	58.599171	140714.500000

```
  Avg CV (2013-2022): 58.18%
  Avg CV (2023-2024): 57.33%
  Std Dev Change (2022→2024): -0.77%

CLINICAL PROFESSORS
--------------------------------------------------------------------------------
 Year  Count          Mean           Std       CV%           IQR
 2013   1324 350712.793956 228760.179629 65.227213 241035.283990
 2014   1476 358723.185427 236980.701705 66.062276 237470.423836
 2015   1576 378119.352777 242497.367019 64.132493 254181.847258
 2016   1648 391538.915926 256751.874066 65.575059 266981.755065
 2017   1711 397143.457317 260869.907212 65.686568 267739.974929
 2018   1792 401294.800005 257341.440031 64.127778 262985.814062
 2019   1766 413843.528137 265633.113775 64.186847 264039.821548
 2020   1890 408756.053989 253595.536407 62.040803 251701.463400
 2021   2154 415485.396567 268798.214797 64.694985 261055.913156
 2022   2308 408832.216171 256462.779877 62.730570 259102.771155
 2023   2466 414806.651010 269361.876714 64.936730 262226.670000
 2024   2524 371263.788431 243612.721270 65.617151 270584.000000

  Avg CV (2013-2022): 64.45%
  Avg CV (2023-2024): 65.28%
  Std Dev Change (2022→2024): -5.01%

RESEARCH PROFESSORS
--------------------------------------------------------------------------------
 Year  Count          Mean           Std       CV%           IQR
 2013    573 292224.630386 192149.110267 65.753907 161872.163940
 2014    582 292791.815201 193865.068047 66.212598 164755.952981
 2015    560 316982.219684 215267.008780 67.911383 175562.949359
 2016    561 316564.444530 221142.241140 69.856942 170003.690319
 2017    564 326491.114545 227151.387271 69.573528 175063.042626
 2018    568 321385.228230 222100.988702 69.107404 178052.039333
 2019    541 330368.001415 232720.400302 70.442779 188860.995872
 2020    576 328089.479985 215979.385579 65.829415 188843.084573
 2021    600 329775.516857 233519.931572 70.811785 190723.693978
 2022    602 321198.697124 216452.637153 67.389015 172284.039997
 2023    633 330493.742227 215626.859412 65.243855 177604.960000
 2024    601 289026.344426 186296.716024 64.456656 208552.000000

  Avg CV (2013-2022): 68.29%
  Avg CV (2023-2024): 64.85%
  Std Dev Change (2022→2024): -13.93%
```

7 Visualization: Rank Variation

Visual trends in variation metrics by rank.

[99]:
```
# Visualize salary variation trends by rank
print("\n" + "=" * 80)
print("VISUALIZATION: SALARY VARIATION TRENDS BY RANK")
print("=" * 80)

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Plot 1: Coefficient of Variation (CV) over time by rank
for rank in ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']:
    rank_data = variation_df[variation_df['Rank'] == rank].sort_values('Year')
    if len(rank_data) > 0:
        axes[0, 0].plot(rank_data['Year'], rank_data['CV%'], marker='o',␣
 ↪label=rank, linewidth=2)

axes[0, 0].axvline(x=2023, color='red', linestyle='--', alpha=0.7, linewidth=2,␣
 ↪label='2023 Threshold')
axes[0, 0].set_title('Coefficient of Variation by Rank (Lower = Less␣
 ↪Dispersion)', fontsize=12, fontweight='bold')
axes[0, 0].set_xlabel('Year')
axes[0, 0].set_ylabel('CV (%)')
axes[0, 0].legend(loc='best')
axes[0, 0].grid(True, alpha=0.3)

# Plot 2: Standard Deviation over time by rank
for rank in ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']:
    rank_data = variation_df[variation_df['Rank'] == rank].sort_values('Year')
    if len(rank_data) > 0:
        axes[0, 1].plot(rank_data['Year'], rank_data['Std'], marker='s',␣
 ↪label=rank, linewidth=2)

axes[0, 1].axvline(x=2023, color='red', linestyle='--', alpha=0.7, linewidth=2,␣
 ↪label='2023 Threshold')
axes[0, 1].set_title('Standard Deviation of Salary by Rank', fontsize=12,␣
 ↪fontweight='bold')
axes[0, 1].set_xlabel('Year')
axes[0, 1].set_ylabel(salary_label)
axes[0, 1].legend(loc='best')
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].ticklabel_format(style='plain', axis='y')

# Plot 3: Interquartile Range (IQR) over time by rank
for rank in ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']:
    rank_data = variation_df[variation_df['Rank'] == rank].sort_values('Year')
```

```
    if len(rank_data) > 0:
        axes[1, 0].plot(rank_data['Year'], rank_data['IQR'], marker='^',␣
 ↪label=rank, linewidth=2)

axes[1, 0].axvline(x=2023, color='red', linestyle='--', alpha=0.7, linewidth=2,␣
 ↪label='2023 Threshold')
axes[1, 0].set_title('Interquartile Range (IQR) by Rank', fontsize=12,␣
 ↪fontweight='bold')
axes[1, 0].set_xlabel('Year')
axes[1, 0].set_ylabel(salary_label)
axes[1, 0].legend(loc='best')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].ticklabel_format(style='plain', axis='y')

# Plot 4: Year-over-year % change in CV for each rank (post vs pre-2023)
cv_change_data = []
ranks_with_data = []

for rank in ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']:
    rank_data_2013_2022 = variation_df[(variation_df['Rank'] == rank) &␣
 ↪(variation_df['Year'] < 2023)]['CV%'].mean()
    rank_data_2023_2024 = variation_df[(variation_df['Rank'] == rank) &␣
 ↪(variation_df['Year'] >= 2023)]['CV%'].mean()

    if not np.isnan(rank_data_2013_2022) and not np.isnan(rank_data_2023_2024):
        change = ((rank_data_2023_2024 - rank_data_2013_2022) /␣
 ↪rank_data_2013_2022 * 100)
        cv_change_data.append(change)
        ranks_with_data.append(rank)

colors = ['red' if x < 0 else 'green' for x in cv_change_data]
axes[1, 1].bar(ranks_with_data, cv_change_data, color=colors, alpha=0.7,␣
 ↪edgecolor='black')
axes[1, 1].axhline(y=0, color='black', linestyle='-', linewidth=0.8)
axes[1, 1].set_title('CV Change: Pre-2023 vs 2023-2024\n(Negative =␣
 ↪Compression)', fontsize=12, fontweight='bold')
axes[1, 1].set_ylabel('% Change in CV')
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\n  Variation analysis complete!")
```

```
================================================================================
VISUALIZATION: SALARY VARIATION TRENDS BY RANK
```

```
================================================================================
```

```
Variation analysis complete!
```

8 Statistical Test: Levene's Variance Test

Assess variance changes pre-2023 vs 2023–2024.

[100]:
```
# Statistical test: Levene's test for equality of variances pre-2023 vs␣
↪2023-2024
from scipy import stats

print("\n" + "=" * 80)
print("STATISTICAL TEST: LEVENE'S TEST FOR VARIANCE EQUALITY")
print("Testing if salary variation within each rank changed significantly")
print("=" * 80)

results_levene = []

for rank in ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']:
```

```python
    # Get pre-2023 and 2023-2024 data
    pre_2023_data = []
    post_2023_data = []

    for year_str in sorted(prof_subsets.keys()):
        year_int = int(year_str)
        df_year = prof_subsets[year_str]
        rank_salaries = df_year[df_year['Rank'] == rank][salary_col].values

        if len(rank_salaries) > 0:
            if year_int < 2023:
                pre_2023_data.extend(rank_salaries)
            else:
                post_2023_data.extend(rank_salaries)

    if len(pre_2023_data) > 1 and len(post_2023_data) > 1:
        # Levene's test
        statistic, p_value = stats.levene(pre_2023_data, post_2023_data)

        # Also calculate variance ratio
        var_pre = np.var(pre_2023_data, ddof=1)
        var_post = np.var(post_2023_data, ddof=1)
        var_ratio = var_post / var_pre

        results_levene.append({
            'Rank': rank,
            'Pre-2023 Var': var_pre,
            'Post-2023 Var': var_post,
            'Var Ratio': var_ratio,
            'Levene Stat': statistic,
            'p-value': p_value,
            'Significant ( =0.05)': 'Yes' if p_value < 0.05 else 'No',
        })

levene_df = pd.DataFrame(results_levene)
print("\n" + levene_df.to_string(index=False))

print("\n" + "=" * 80)
print("INTERPRETATION")
print("=" * 80)
print("\nVariance Ratio > 1: Variance INCREASED (2023-2024 more disperse)")
print("Variance Ratio < 1: Variance DECREASED (2023-2024 more compressed)")
print("p-value < 0.05: Change in variance is STATISTICALLY SIGNIFICANT")
print("\n" + "-" * 80)

for _, row in levene_df.iterrows():
    print(f"\n{row['Rank'].upper()} PROFESSORS:")
```

```
    print(f"  Variance 2013-2022: ${row['Pre-2023 Var']:,.0f}")
    print(f"  Variance 2023-2024: ${row['Post-2023 Var']:,.0f}")
    print(f"  Variance Ratio (2023-24 / 2013-22): {row['Var Ratio']:.3f}")

    if row['Var Ratio'] < 1:
        pct_compression = ((1 - row['Var Ratio']) * 100)
        print(f"  → COMPRESSION: Variance decreased by {pct_compression:.1f}%")
    else:
        pct_increase = ((row['Var Ratio'] - 1) * 100)
        print(f"  → EXPANSION: Variance increased by {pct_increase:.1f}%")

    print(f"  Levene p-value: {row['p-value']:.4f} {'  SIGNIFICANT' if␣
↪row['p-value'] < 0.05 else '(not significant)'}")
```

```
================================================================================
STATISTICAL TEST: LEVENE'S TEST FOR VARIANCE EQUALITY
Testing if salary variation within each rank changed significantly
================================================================================

     Rank  Pre-2023 Var  Post-2023 Var  Var Ratio  Levene Stat  p-value
Significant ( =0.05)
Assistant  2.201961e+10   2.323921e+10   1.055387     2.338093 0.126249
No
Associate  3.301428e+10   2.980544e+10   0.902805     2.168449 0.140875
No
     Full  2.611151e+10   2.425120e+10   0.928755     0.468062 0.493882
No
 Clinical  6.511994e+10   6.633552e+10   1.018667     0.473440 0.491416
No
 Research  4.736000e+10   4.115069e+10   0.868891     0.359481 0.548814
No

================================================================================
INTERPRETATION
================================================================================

Variance Ratio > 1: Variance INCREASED (2023-2024 more disperse)
Variance Ratio < 1: Variance DECREASED (2023-2024 more compressed)
p-value < 0.05: Change in variance is STATISTICALLY SIGNIFICANT

--------------------------------------------------------------------------------

ASSISTANT PROFESSORS:
  Variance 2013-2022: $22,019,611,768
  Variance 2023-2024: $23,239,205,942
  Variance Ratio (2023-24 / 2013-22): 1.055
  → EXPANSION: Variance increased by 5.5%
```

```
  Levene p-value: 0.1262 (not significant)

ASSOCIATE PROFESSORS:
  Variance 2013-2022: $33,014,277,063
  Variance 2023-2024: $29,805,444,492
  Variance Ratio (2023-24 / 2013-22): 0.903
  → COMPRESSION: Variance decreased by 9.7%
  Levene p-value: 0.1409 (not significant)

FULL PROFESSORS:
  Variance 2013-2022: $26,111,511,747
  Variance 2023-2024: $24,251,197,735
  Variance Ratio (2023-24 / 2013-22): 0.929
  → COMPRESSION: Variance decreased by 7.1%
  Levene p-value: 0.4939 (not significant)

CLINICAL PROFESSORS:
  Variance 2013-2022: $65,119,939,935
  Variance 2023-2024: $66,335,516,936
  Variance Ratio (2023-24 / 2013-22): 1.019
  → EXPANSION: Variance increased by 1.9%
  Levene p-value: 0.4914 (not significant)

RESEARCH PROFESSORS:
  Variance 2013-2022: $47,359,999,681
  Variance 2023-2024: $41,150,694,590
  Variance Ratio (2023-24 / 2013-22): 0.869
  → COMPRESSION: Variance decreased by 13.1%
  Levene p-value: 0.5488 (not significant)

     Rank  Pre-2023 Var  Post-2023 Var  Var Ratio  Levene Stat  p-value
Significant ( =0.05)
Assistant  2.201961e+10   2.323921e+10   1.055387     2.338093 0.126249
No
Associate  3.301428e+10   2.980544e+10   0.902805     2.168449 0.140875
No
     Full  2.611151e+10   2.425120e+10   0.928755     0.468062 0.493882
No
 Clinical  6.511994e+10   6.633552e+10   1.018667     0.473440 0.491416
No
 Research  4.736000e+10   4.115069e+10   0.868891     0.359481 0.548814
No

================================================================================
INTERPRETATION
================================================================================

Variance Ratio > 1: Variance INCREASED (2023-2024 more disperse)
```

```
Variance Ratio < 1: Variance DECREASED (2023-2024 more compressed)
p-value < 0.05: Change in variance is STATISTICALLY SIGNIFICANT

--------------------------------------------------------------------------------

ASSISTANT PROFESSORS:
  Variance 2013-2022: $22,019,611,768
  Variance 2023-2024: $23,239,205,942
  Variance Ratio (2023-24 / 2013-22): 1.055
  → EXPANSION: Variance increased by 5.5%
  Levene p-value: 0.1262 (not significant)

ASSOCIATE PROFESSORS:
  Variance 2013-2022: $33,014,277,063
  Variance 2023-2024: $29,805,444,492
  Variance Ratio (2023-24 / 2013-22): 0.903
  → COMPRESSION: Variance decreased by 9.7%
  Levene p-value: 0.1409 (not significant)

FULL PROFESSORS:
  Variance 2013-2022: $26,111,511,747
  Variance 2023-2024: $24,251,197,735
  Variance Ratio (2023-24 / 2013-22): 0.929
  → COMPRESSION: Variance decreased by 7.1%
  Levene p-value: 0.4939 (not significant)

CLINICAL PROFESSORS:
  Variance 2013-2022: $65,119,939,935
  Variance 2023-2024: $66,335,516,936
  Variance Ratio (2023-24 / 2013-22): 1.019
  → EXPANSION: Variance increased by 1.9%
  Levene p-value: 0.4914 (not significant)

RESEARCH PROFESSORS:
  Variance 2013-2022: $47,359,999,681
  Variance 2023-2024: $41,150,694,590
  Variance Ratio (2023-24 / 2013-22): 0.869
  → COMPRESSION: Variance decreased by 13.1%
  Levene p-value: 0.5488 (not significant)
```

9 Visualization: Distribution Comparison

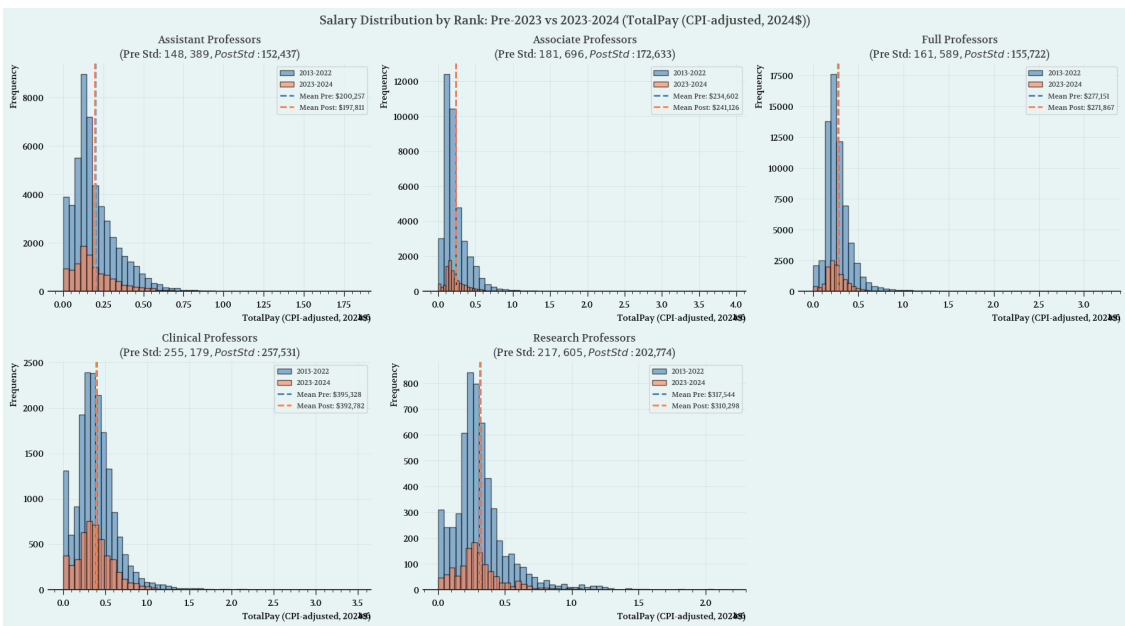
Pre-2023 vs 2023–2024 salary distributions by rank.

[101]:
```python
# Visualize distribution comparison: pre-2023 vs 2023-2024
print("\n" + "=" * 80)
print("DISTRIBUTION COMPARISON: PRE-2023 vs 2023-2024")
```

```
print("=" * 80)

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle(f'Salary Distribution by Rank: Pre-2023 vs 2023-2024␣
↪({salary_label})', fontsize=14, fontweight='bold')

ranks_to_plot = ['Assistant', 'Associate', 'Full', 'Clinical', 'Research']
colors_pre = ['steelblue', 'steelblue', 'steelblue', 'steelblue', 'steelblue']
colors_post = ['coral', 'coral', 'coral', 'coral', 'coral']

for idx, rank in enumerate(ranks_to_plot):
    ax = axes.flat[idx]

    # Collect pre-2023 and post-2023 data
    pre_2023_data = []
    post_2023_data = []

    for year_str in sorted(prof_subsets.keys()):
        year_int = int(year_str)
        df_year = prof_subsets[year_str]
        rank_salaries = df_year[df_year['Rank'] == rank][salary_col].values

        if len(rank_salaries) > 0:
            if year_int < 2023:
                pre_2023_data.extend(rank_salaries)
            else:
                post_2023_data.extend(rank_salaries)

    if len(pre_2023_data) > 0 and len(post_2023_data) > 0:
        # Plot distributions
        ax.hist(pre_2023_data, bins=50, alpha=0.6, color='steelblue',␣
↪label='2013-2022', edgecolor='black')
        ax.hist(post_2023_data, bins=50, alpha=0.6, color='coral',␣
↪label='2023-2024', edgecolor='black')

        # Add statistics
        mean_pre = np.mean(pre_2023_data)
        mean_post = np.mean(post_2023_data)
        std_pre = np.std(pre_2023_data)
        std_post = np.std(post_2023_data)

        ax.axvline(mean_pre, color='steelblue', linestyle='--', linewidth=2,␣
↪label=f'Mean Pre: ${mean_pre:,.0f}')
        ax.axvline(mean_post, color='coral', linestyle='--', linewidth=2,␣
↪label=f'Mean Post: ${mean_post:,.0f}')
```

```
        ax.set_title(f'{rank} Professors\n(Pre Std: ${std_pre:,.0f}, Post Std:␣
↪${std_post:,.0f})', fontweight='bold')
        ax.set_xlabel(salary_label)
        ax.set_ylabel('Frequency')
        ax.legend(fontsize=8)
        ax.grid(True, alpha=0.3)

# Remove empty subplot
axes.flat[-1].axis('off')

plt.tight_layout()
plt.show()

print("\n Distribution comparison complete!")
```

```
================================================================================
DISTRIBUTION COMPARISON: PRE-2023 vs 2023-2024
================================================================================
```




```
 Distribution comparison complete!
```

10 Analysis: 2023 Leavers (vs 2024)

Professors present in 2023 but not in 2024.

[102]:
```
# Identify professors present in 2023 but not in 2024
df_2023 = df23.copy()
df_2024 = df24.copy()

key_cols = ['EmployerName', 'Position', 'RegularPay', 'TotalWages']
available_cols = [col for col in key_cols if col in df_2023.columns and col in␣
 ↪df_2024.columns]

if len(available_cols) < len(key_cols):
    missing = list(set(key_cols) - set(available_cols))
    raise ValueError(f"Missing required columns for matching: {missing}")

# Record-level match using available fields (no unique person identifier in␣
 ↪dataset)
left_merge = df_2023.merge(df_2024[available_cols].drop_duplicates(),␣
 ↪on=available_cols, how='left', indicator=True)
left_2023 = left_merge[left_merge['_merge'] == 'left_only'].
 ↪drop(columns=['_merge'])

# Rank classification (reuse existing if defined)
if 'categorize_rank' in globals():
    rank_func = categorize_rank
else:
    def rank_func(position):
        pos_lower = position.lower()
        if 'asst' in pos_lower or 'assistant' in pos_lower:
            return 'Assistant'
        elif 'assoc' in pos_lower:
            return 'Associate'
        elif 'clin' in pos_lower:
            return 'Clinical'
        elif 'res' in pos_lower or 'research' in pos_lower:
            return 'Research'
        else:
            return 'Full'

left_2023 = left_2023.copy()
left_2023['Rank'] = left_2023['Position'].apply(rank_func)

# Summaries
left_by_employer = (left_2023.groupby('EmployerName')
                    .size()
                    .sort_values(ascending=False)
                    .head(15)
                    .reset_index(name='LeftCount'))

left_by_rank = (left_2023.groupby('Rank')
```

```
                .size()
                .sort_values(ascending=False)
                .reset_index(name='LeftCount'))

salary_summary = left_2023[['RegularPay', 'TotalWages']].agg(['count', 'mean',␣
 ↪'median', 'min', 'max']).T

from IPython.display import display

left_2023_count = len(left_2023)
display(pd.DataFrame({'Left_2023_Count': [left_2023_count]}))
display(left_by_employer)
display(left_by_rank)
display(salary_summary)
```

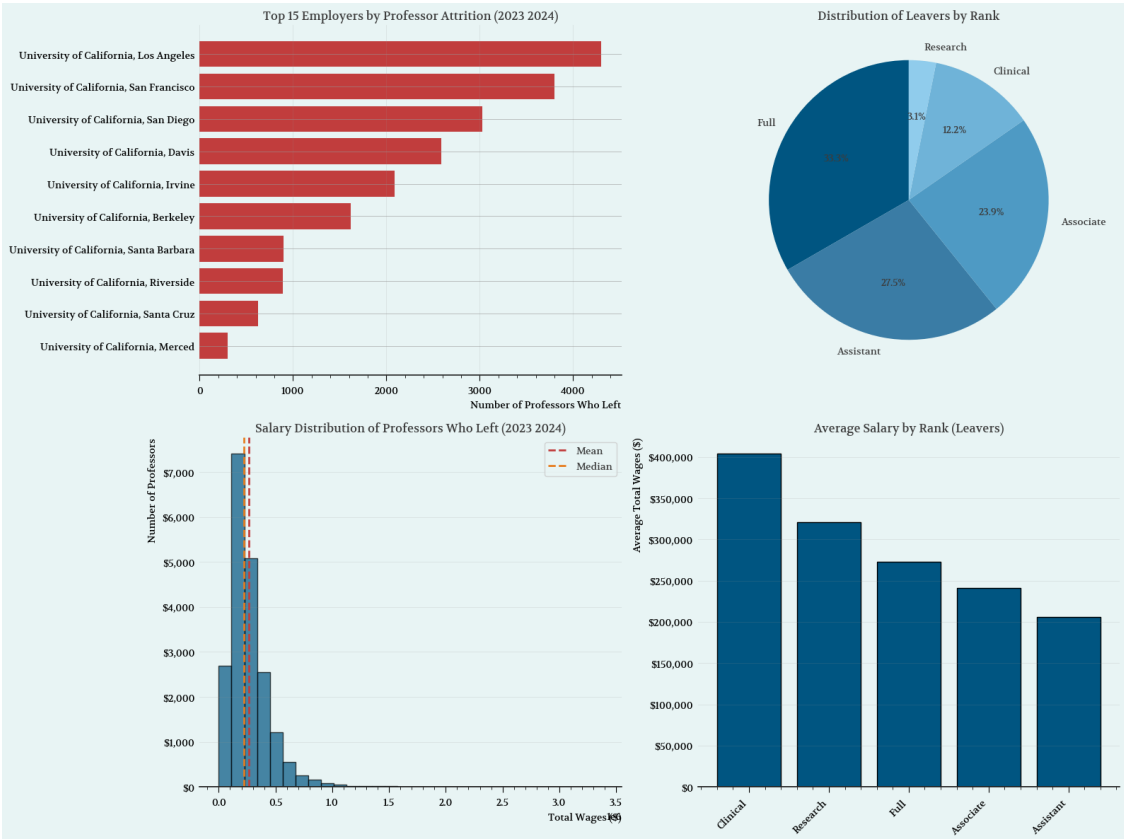
```
   Left_2023_Count
0            20152

                              EmployerName  LeftCount
0    University of California, Los Angeles       4302
1  University of California, San Francisco       3804
2      University of California, San Diego       3027
3          University of California, Davis       2587
4         University of California, Irvine       2089
5       University of California, Berkeley       1621
6  University of California, Santa Barbara        902
7      University of California, Riverside        889
8     University of California, Santa Cruz        630
9         University of California, Merced        301

        Rank  LeftCount
0       Full       6715
1  Assistant       5535
2  Associate       4809
3   Clinical       2460
4   Research        633

               count           mean    median  min        max
RegularPay  20152.0  167716.252382  158161.5  0.0   801717.0
TotalWages  20152.0  264470.629764  224761.0  3.0  3377514.0
```

10.1 Visualization: 2023 Leavers

[103]:
```
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# 1. Top 15 Employers by Leaver Count
ax1 = axes[0, 0]
ax1.barh(range(len(left_by_employer)), left_by_employer['LeftCount'],␣
 ↪color='#c13d3d')
```

```
ax1.set_yticks(range(len(left_by_employer)))
ax1.set_yticklabels(left_by_employer['EmployerName'])
ax1.invert_yaxis()
ax1.set_xlabel('Number of Professors Who Left')
ax1.set_title('Top 15 Employers by Professor Attrition (2023→2024)',␣
 ↪fontweight='bold')
ax1.grid(axis='x', alpha=0.3)

# 2. Leavers by Rank (Pie Chart)
ax2 = axes[0, 1]
colors_pie = ['#005581', '#3a7ca5', '#4e9ac4', '#6fb3d8', '#90ccec']
ax2.pie(left_by_rank['LeftCount'], labels=left_by_rank['Rank'], autopct='%1.
 ↪1f%%',
        startangle=90, colors=colors_pie[:len(left_by_rank)])
ax2.set_title('Distribution of Leavers by Rank', fontweight='bold')

# 3. Salary Distribution (TotalWages)
ax3 = axes[1, 0]
ax3.hist(left_2023['TotalWages'], bins=30, color='#005581', edgecolor='black',␣
 ↪alpha=0.7)
ax3.axvline(left_2023['TotalWages'].mean(), color='#c13d3d', linestyle='--',␣
 ↪linewidth=2, label='Mean')
ax3.axvline(left_2023['TotalWages'].median(), color='#e67e22', linestyle='--',␣
 ↪linewidth=2, label='Median')
ax3.set_xlabel('Total Wages ($)')
ax3.set_ylabel('Number of Professors')
ax3.set_title('Salary Distribution of Professors Who Left (2023→2024)',␣
 ↪fontweight='bold')
ax3.legend()
ax3.grid(axis='y', alpha=0.3)

# 4. Average Salary by Rank for Leavers
ax4 = axes[1, 1]
rank_avg_salary = left_2023.groupby('Rank')['TotalWages'].mean().
 ↪sort_values(ascending=False)
ax4.bar(range(len(rank_avg_salary)), rank_avg_salary.values, color='#005581',␣
 ↪edgecolor='black')
ax4.set_xticks(range(len(rank_avg_salary)))
ax4.set_xticklabels(rank_avg_salary.index, rotation=45, ha='right')
ax4.set_ylabel('Average Total Wages ($)')
ax4.set_title('Average Salary by Rank (Leavers)', fontweight='bold')
ax4.grid(axis='y', alpha=0.3)

# Format y-axis with commas for salary values
for ax in [ax3, ax4]:
```

```
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'${x:,.0f}' if␣
↪ax == ax4 else f'{x:,.0f}'))

plt.tight_layout()
plt.show()
```



[ ]:

```

```